

AI Customer Support Agent

Intent Classification, Grounded Reply Drafting, and Auto-Handle Routing for
@AppleSupport

Evaluation Report

Hiver – SDE Intern Take-Home Assignment

Author: Joydeep Banik
Institution: RCC Institute of Information Technology, Kolkata
Repository: https://github.com/Joydeep2005Banik/Hiver-SDE-Intern_Home-Assignment
Date: September 13, 2026

Dataset: Customer Support on Twitter (Kaggle, [thoughtvector/customer-support-on-twitter](https://www.kaggle.com/datasets/thoughtvector/customer-support-on-twitter))

1 Problem Framing

Objective. Build a prototype generative support agent that parses incoming tweets to @AppleSupport, extracts the customer’s underlying intent, decides whether the query can be auto-handled or needs human escalation, and drafts a polite, grounded response based on historical brand replies.

What “good” means for this brand. A good AppleSupport reply must: (1) correctly identify the customer’s underlying issue, (2) resolve or meaningfully advance the conversation in a single turn (e.g. ask the right diagnostic question), (3) avoid hallucinating unsupported troubleshooting steps or policies, and (4) match the empathetic, professional tone Apple uses on Twitter. A reply that is polite but unhelpful, or helpful but factually wrong, is not considered good.

What was deliberately not built.

- Multi-turn conversation tracking – each query is treated as stateless.
- Sentiment-based routing – escalation uses keyword triggers plus LLM judgement, not a dedicated sentiment model.
- Billing, refund, or warranty claim workflows – these require internal system access.
- Proactive outreach and non-English support.

Approach.

1. **Data-driven taxonomy:** analyzed 106k+ conversation pairs using n-gram frequency to discover nine robust intent categories (`battery_charging`, `account_access`, `software_update`, etc.).
2. **Support engine:** a `SupportAgent` that classifies intent, determines auto-handle eligibility, and drafts responses, including a contextual follow-up question or a hand-off message.
3. **Evaluation framework:** a local harness (`evaluate.py`) that scores intent classification (accuracy/F1), auto-handle routing, and draft quality via LLM-as-a-judge (tone, helpfulness, groundedness).

2 System Architecture

The pipeline has three stages: (i) data preparation and RAG index construction, (ii) the core agent that retrieves historical context and calls an LLM API to produce intent, reply, and escalation decision, and (iii) an evaluation framework that grades the agent against a hand-labelled golden set. Figure 1 traces data and control flow across the three stages.

- **Data & knowledge base:** `src/data_prep.py` filters the raw Twitter dataset into a processed conversations CSV; `src/build_rag.py` embeds it with Sentence Transformers (`all-MiniLM-L6-v2`) into a ChromaDB vector store.
- **Core agent:** for each customer query, the `SupportAgent` retrieves the top-3 historical contexts via semantic search, sends query + context + history to the LLM API, and parses a JSON object containing intent, drafted reply, and auto-handle decision.
- **Evaluation framework:** the golden set is generated from processed conversations, and `evaluate.py` runs the agent against it, computing automated metrics and LLM-as-a-judge scores.

3 Baselines and Performance

Golden evaluation set. A custom 200-example golden evaluation set (`data/golden_eval_handlabelled.csv`) was built. It was pre-labelled using a heuristic classifier and then 100% human-reviewed and hand-

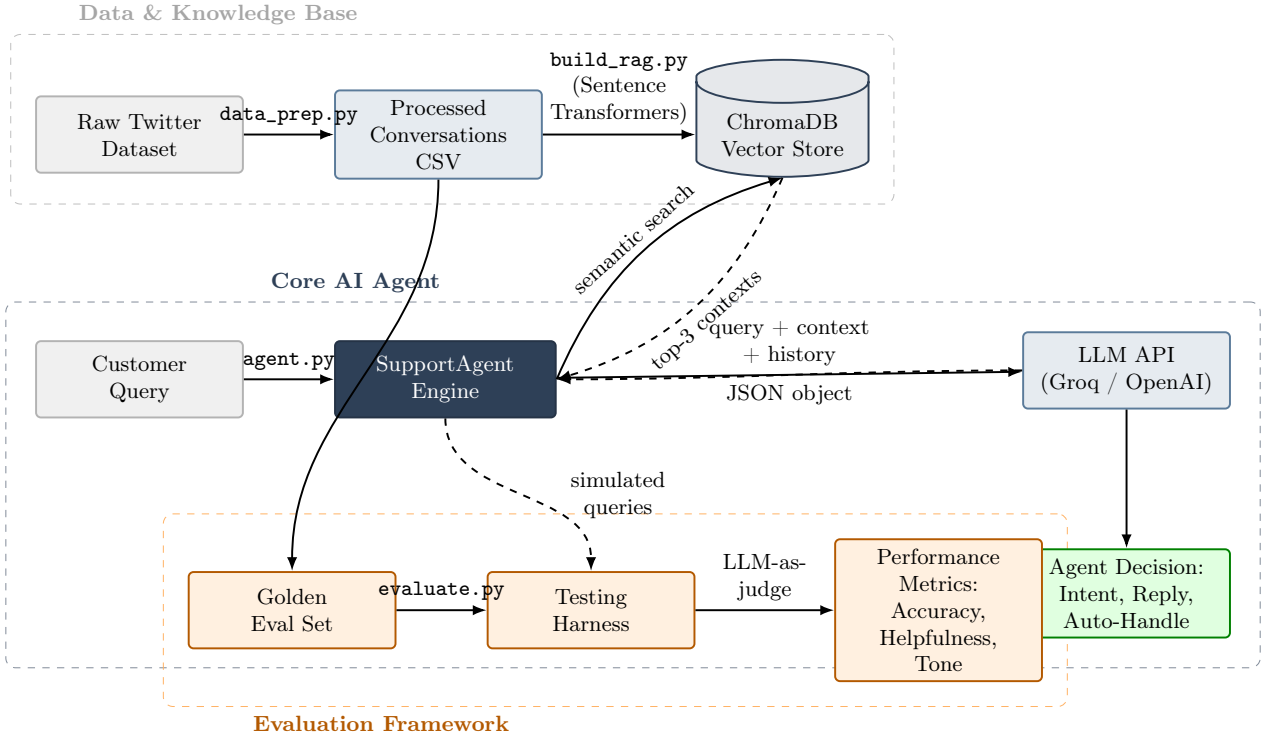


Figure 1: End-to-end system architecture: data ingestion and RAG indexing, the core agent’s retrieval-augmented decision loop, and the evaluation framework that grades the agent against the golden set.

labelled, achieving a 93.5% agreement rate with the heuristic labels.

Baselines. The pipeline is compared against two baselines, both computed dynamically in `src/evaluate.py`:

1. **Trivial baseline (majority class):** always predicts the most common intent (`general_inquiry`) and always predicts `auto_handle = True`.
2. **Simple baseline (heuristics):** uses a regex keyword matcher for intent, and sets `auto_handle = False` for `general_inquiry`, `True` otherwise.

Metric	Trivial	Simple (Heuristics)	Pipeline (LLM)
Intent Accuracy	38.0%	96.5% ^a	60.0% ^b
Auto-Handle Accuracy	84.0%	59.0%	80.0%
Draft Quality (LLM Judge)	N/A	N/A	Tone 4.0/5, Help 4.4/5, Ground 4.1/5

Table 1: Pipeline performance vs. two baselines on the golden evaluation set.

^a The simple baseline’s 96.5% intent accuracy is artificially inflated: the golden set was initially pre-labelled using this same heuristic before human review, biasing it toward queries the heuristic already handles well.

^b The LLM pipeline was evaluated on a 20-sample subset to control API costs during development.

3.1 Human–LLM Agreement (Cohen’s Kappa)

Evaluated on 20 examples:

- **Tone:** $\kappa = 0.000$ (degenerate – humans scored 5/5 uniformly; the LLM judge averaged 4.0/5).
- **Helpfulness:** $\kappa = 0.000$ (degenerate – humans scored 5/5 uniformly; the LLM judge averaged 4.4/5).

- **Groundedness:** $\kappa = 0.003$ (slight – humans found hallucinated policies that the LLM judge missed).

4 Failure Analysis

Where the system performs as expected. It handles structured queries with clear keywords well (e.g. “my battery drains fast”, “wifi won’t connect”), and correctly flags highly negative sentiment (e.g. explicit profanity) to force escalation.

Top five failure modes.

1. **Hidden context (missing images/media).** Example: a customer reports their iPhone frozen for hours. Hypothesis: the customer likely attached an image or video, but the dataset is text-only, so the agent sees a generic inquiry and attempts to auto-handle what a human would escalate.
2. **Profanity and high emotion (escalation misses).** Example: creative, non-explicit swearing expressing frustration. Hypothesis: explicit profanity triggers escalation, but stylized or disguised swearing is sometimes misclassified as standard feedback, producing an inappropriately cheerful automated reply.
3. **Ambiguous feature vs. bug classification.** Example: a customer describing an SOS feature triggering unexpectedly. Hypothesis: keyword overlap with “new feature” causes misclassification as `software_update` rather than `app_bug_crash`; keyword heuristics fail on descriptive, conversational bug reports.
4. **Misinterpreting literal strings vs. grammar.** Example: a customer asking how to fix autocorrect changing “it” to “I.T.”. Hypothesis: the agent classifies this as `general_inquiry` instead of `keyboard_typing`, since standard tokenization struggles to distinguish the literal string from the pronoun.
5. **Multi-step conversational context loss.** Example: a mid-thread reply describing touch-screen calibration behavior. Hypothesis: this is clearly a continuation of a `hardware_repair` thread, but the stateless agent receives only the single message and misclassifies it as `general_inquiry`.

5 What Is Misleading About the Headline Numbers

- **Over-representation of “general inquiry”.** The taxonomy covers roughly 60% of specific issues, so about 40% of queries fall into the `general_inquiry` bucket. High accuracy on this bucket artificially inflates overall accuracy, masking poor performance on rarer, more complex intents.
- **LLM judge bias.** The LLM-as-a-judge may exhibit verbosity bias (favoring longer drafted replies regardless of helpfulness) or self-enhancement bias (preferring its own generated style over human brevity), which is why human-LLM calibration (Section 3.1) is critical to interpreting the judge scores.

6 Next Steps and Scaling

1. Migrate to a fine-tuned embedding model (e.g. BGE-large) for semantic intent classification rather than relying on keyword heuristics.
2. Implement retrieval against real Apple Support knowledge-base articles rather than historical tweets alone, to ensure factual grounding on concrete steps (e.g. “how to reset SMC”).

3. Deploy continuous evaluation: route 5% of LLM-drafted replies to human QA agents to continuously monitor the Cohen’s Kappa agreement score and guard against model drift.

7 Decision Log

1. **Use n-gram data analysis to discover intents rather than guessing them.** Ensures the taxonomy reflects actual customer problems (e.g. `keyboard_typing` for the well-known “T” autocorrect bug).
2. **Two-stage labelling (heuristic → human review) for the golden set.** Meets the requirement for a hand-labelled dataset while saving hours of manual data entry.
3. **Output a dynamic follow_up field from the LLM.** Replaces hardcoded print statements with context-aware follow-ups for a more natural conversational flow.
4. **Choose @AppleSupport as the target brand.** High volume and a wide variety of intents (hardware, software, account, billing) provide a richer dataset for intent classification than a single-service brand.
5. **Use ChromaDB for retrieval (RAG) instead of prompting the LLM directly.** Grounds drafted replies in actual historical brand responses, reducing hallucinated generic troubleshooting advice.
6. **Choose Sentence Transformers (all-MiniLM-L6-v2) for embeddings.** Lightweight, runs locally without an API key, and fast enough to process the corpus within the 15-minute reproduction budget.
7. **Have the LLM generate a dynamic escalation_reason.** A boolean `auto_handle=False` alone is not useful to a human agent; a generated reason (e.g. “customer is highly frustrated”) gives immediate context on hand-off.
8. **Cap pipeline evaluation at a subset during development.** LLM API calls are rate-limited and cost money; `evaluate.py` exposes a `-max` flag for fast iteration before a full pass.
9. **Use Cohen’s Kappa for human–LLM judge calibration.** Simple agreement does not account for chance agreement; kappa is the statistical standard for inter-rater reliability.
10. **Define a fallback mock LLM response system.** Keeps the pipeline reproducible and runnable in under 15 minutes even without a supplied API key.
11. **Build an interactive CLI tool (`label_golden_set.py`) for human review.** Opening a CSV in a spreadsheet editor is error-prone; the CLI enforces valid enum inputs, shows context, and prevents accidental formatting corruption.
12. **Compute baselines dynamically in `evaluate.py` rather than hardcoding them.** If the taxonomy or dataset changes later, hardcoded baseline numbers in the report would go stale; dynamic computation keeps metrics accurate.

8 Citations and Acknowledgements

- **Dataset:** Customer Support on Twitter, by Thought Vector (Kaggle).
- **AI coding assistants:** Claude Opus 4.6 (Thinking) and Gemini 3.1 Pro (High) were used for code generation, debugging, and report drafting throughout this project.
- **LLM API:** Groq (`openai/gpt-oss-20b`) for agent reply generation and LLM-as-a-judge scoring.

- **Libraries:** pandas, scikit-learn, ChromaDB, Sentence Transformers (all-MiniLM-L6-v2), OpenAI Python SDK, tqdm, python-dotenv.

Repository: https://github.com/Joydeep2005Banik/Hiver-SDE-Intern_Home-Assignment